

Persona V3 - VPS Deployment & API Blueprint

Official architectural specification for **Persona V3** (https://github.com/BawanthaBeliwaththa/Persona_V3). Covers both **Terminal-Free (GUI/PaaS)** and **Terminal (SSH/Systemd/Nginx)** deployment methods, plus frontend team API connection standards.

Method A: Zero-Terminal VPS Deployment for Persona V3

Deploy **Persona V3** to a VPS without manually running terminal SSH commands using automated browser-based PaaS platforms:



1. Coolify (PaaS)

Recommended GUI Method for

Persona V3. Self-hosted web dashboard for managing Docker apps and VPS servers.

- One-click VPS marketplace install (DigitalOcean/Hetzner).
- Connects directly to `https://github.com/BawanthaBeliwat`
- Auto-installs Docker & Playwright browser binaries.
- Automatic SSL (HTTPS) certificate generation.



2. GitHub Actions CI/CD

Automate Persona V3 deployment on every `git push` using GitHub's build runner.

- You push code updates to `BawanthaBeliwaththa/Persona_V3`.
- GitHub Actions SSHs into VPS silently in background.
- Builds Docker image & restarts service automatically.
- Zero terminal interaction after initial workflow creation.



3. Managed Cloud Platforms

Managed hosting platforms like `Render`, `Railway`, or `Fly.io`.

- 100% web browser managed interface.
- No server operating system management.
- Auto-detects Dockerfile / Persona V3 Flask app.
- Includes free HTTPS domain endpoints.

| Step-by-Step: Coolify GUI Setup for Persona V3

1. **Create VPS Instance:** Launch an Ubuntu VPS (Hetzner / DigitalOcean / Vultr) with minimum 2GB RAM. Select the *Coolify 1-Click App* option.
2. **Access Web Dashboard:** Open `https://YOUR_VPS_IP:8000` in your web browser and register your admin account.
3. **Connect GitHub:** Go to **Sources** → **GitHub App** and authorize Coolify to access `BawanthaBeliwaththa/Persona_V3` repository.
4. **Create New Resource:** Select `Persona_V3` repository & `main` branch.
5. **Environment Variables:** Set `PYTHONUNBUFFERED=1`, `FLASK_ENV=production`.
6. **Click Deploy:** Coolify builds the Persona V3 container, runs `playwright install chromium`, configures your domain with SSL, and handles future auto-redeployments.

Dockerfile (Placed in Persona_V3 root)

```
FROM mcr.microsoft.com/playwright/python:v1.45.0-noble
```

```
WORKDIR /app
```

```
# Copy Persona V3 requirements & install dependencies
```

```
COPY req.txt .
```

```
RUN pip install --no-cache-dir -r req.txt
```

```
# Install Playwright browser dependencies
```

```
RUN playwright install chromium
```

```
# Copy Persona V3 source code
```

```
COPY . .
```

```
# Expose Persona V3 Flask port
```

```
EXPOSE 5000
```

```
# Start Persona V3 backend with Gunicorn
```

```
CMD ["gunicorn", "--workers=2", "--threads=4", "--bind=0.0.0.0:5000", "app:app"]
```

| Method B: Terminal & SSH VPS Deployment Guide for Persona V3

Deploy **Persona V3** using the Linux SSH command line (Ubuntu 22.04 / 24.04 LTS):



1. Connect & Update Server

Connect to your VPS server terminal via SSH and install Python 3, Git, Nginx, and system build utilities:

```
ssh root@YOUR_VPS_IP
```

```
# Update apt repositories
```

```
sudo apt update && sudo apt upgrade -y
```

```
# Install Python, pip, venv, git, and Nginx
```

```
sudo apt install -y python3 python3-pip python3-venv git curl nginx
```



2. Clone Persona V3 & Setup Virtual Env

Clone the official Persona V3 repository into `/var/www` and set up a Python virtual environment:

```
cd /var/www
```

```
git clone https://github.com/BawanthaBeliwaththa/Persona_V3.git
```

```
cd Persona_V3
```

```
# Create & activate venv
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
# Install Persona V3 requirements
```

```
pip install --upgrade pip
```

```
pip install -r req.txt
```

3. Install Playwright & OS Dependencies via Terminal

Install system dependencies for Playwright inside the Persona V3 virtual environment:

Terminal (Playwright & OS Library Installation)

```
# Install Playwright browser binaries & system OS dependencies automatically
playwright install --with-deps chromium
```

4. Configure Systemd Service (personav3.service)

Create a background system daemon for Persona V3 to ensure continuous uptime and automatic restart on server reboot.

Terminal: `sudo nano /etc/systemd/system/personav3.service`

```
[Unit]
Description=Persona V3 Flask Playwright Service
After=network.target

[Service]
User=root
WorkingDirectory=/var/www/Persona_V3
Environment="PATH=/var/www/Persona_V3/venv/bin"
Environment="PYTHONUNBUFFERED=1"
ExecStart=/var/www/Persona_V3/venv/bin/gunicorn --workers 2 --threads 4 --bind
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Enable and start the service via terminal commands:

```
# Reload systemd manager & start service
sudo systemctl daemon-reload
sudo systemctl start personav3
```

```
sudo systemctl enable personav3
```

```
# Verify service status
```

```
sudo systemctl status personav3
```

5. Configure Nginx Reverse Proxy & SSL (HTTPS)

Configure Nginx to proxy incoming API requests to Persona V3 backend on port 5000, supporting Server-Sent Events (SSE):

Terminal: `sudo nano /etc/nginx/sites-available/personav3`

```
server {  
    listen 80;  
    server_name api.yourdomain.com; # Or your VPS IP address  
  
    location / {  
        proxy_pass http://127.0.0.1:5000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
  
        # Disable buffering for Persona V3 Server-Sent Events (SSE live status  
        proxy_http_version 1.1;  
        proxy_set_header Connection '';  
        proxy_buffering off;  
        proxy_cache off;  
    }  
}
```

Terminal Commands (Activate Nginx & Obtain Free SSL Certificate)

```
# Enable site configuration
```

```
sudo ln -s /etc/nginx/sites-available/personav3 /etc/nginx/sites-enabled/
```

```
sudo nginx -t
```

```
sudo systemctl restart nginx
```

```
# Obtain SSL Certificate via Certbot (HTTPS)
```

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d api.yourdomain.com
```

Persona V3 Terminal Management Cheat Sheet

Action Task	Terminal Command
View Persona V3 Live App Logs	<code>sudo journalctl -u personav3 -f</code>
Restart Persona V3 Service	<code>sudo systemctl restart personav3</code>
View Nginx Server Logs	<code>sudo tail -f /var/log/nginx/error.log</code>
Pull Latest Code & Restart Persona V3	<code>cd /var/www/Persona_V3 && git pull && sudo systemctl restart personav3</code>

Persona V3 Frontend API Integration Architecture

The separate frontend team (React, Next.js, Vue, iOS/Android) will integrate directly with **Persona V3** backend endpoints.



CORS Enabled: `flask_cors` is configured in `app.py` line 54, allowing external frontend apps to make AJAX / `fetch()` requests.

Persona V3 API Endpoint Reference

HTTP Method	Endpoint Route	Request Body Parameters	Description & Expected Response
POST	<code>/api/scrapper/init</code>	<code>{ "headless": true, "browser_type":</code>	Initializes Playwright browser session for Persona V3. Returns

HTTP Method	Endpoint Route	Request Body Parameters	Description & Expected Response
		<code>"chromium", "session_name": "default" }</code>	<code>{"success": true, "message": "Initialized"}.</code>
POST	<code>/api/scrapper/kill-browser</code>	<code>{}</code>	Emergency endpoint to forcefully close browser & kill orphan Chromium processes.
GET	<code>/admin</code>	None	Internal Admin Dashboard template for monitoring live SSE scraper logs.
GET	<code>/api/events</code>	None (EventSource connection)	Server-Sent Events (SSE) stream for real-time progress & status updates.

Authentication & CORS Security Configuration

Restrict allowed origins in production so unauthorized web apps cannot access your Persona V3 backend:

app.py (Production CORS & API Key Middleware for Persona V3)

```
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)

# Restrict CORS to trusted Frontend Domains
ALLOWED_ORIGINS = [
    "https://your-frontend-team-app.com",
    "http://localhost:3000"
]
CORS(app, resources={r"/api/*": {"origins": ALLOWED_ORIGINS}})
```

```
# Persona V3 API Secret Key Protection
API_SECRET_KEY = os.getenv("API_SECRET_KEY", "your-secure-persona-v3-token-123")

@app.before_request
def check_api_key():
    # Allow public index/admin preview, restrict /api/* routes
    if request.path.startswith('/api/'):
        auth_header = request.headers.get("Authorization")
        if not auth_header or auth_header != f"Bearer {API_SECRET_KEY}":
            return jsonify({"success": False, "error": "Unauthorized Persona V3"

```

Frontend Client Request Examples for Persona V3

Share these code snippets with the frontend development team so they can quickly integrate their UI with Persona V3 API endpoints.

1. JavaScript / TypeScript (Fetch API)

api.js (Frontend Service Layer for Persona V3)

```
const API_BASE_URL = "https://api.yourdomain.com"; // Your Persona V3 VPS domain
const API_KEY = "your-secure-persona-v3-token-123";

/**
 * Initialize Persona V3 Scraper Session
 */
export async function initializeScraper() {
    try {
        const response = await fetch(`${API_BASE_URL}/api/scraper/init`, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'Authorization': `Bearer ${API_KEY}`
            },
            body: JSON.stringify({
                headless: true, // VPS must execute in headless mode!
                browser_type: 'chromium',
                session_name: 'client_session_1'
            })
        });
    }

```

```

        const data = await response.json();
        if (!response.ok) throw new Error(data.error || 'Failed to initialize P
        console.log("Persona V3 status:", data);
        return data;
    } catch (err) {
        console.error("Persona V3 API Error:", err);
        throw err;
    }
}

```

2. Listening to Live Progress (Server-Sent Events / SSE)

sse_client.js (Real-time Progress Tracker)

```

// Connect to live Persona V3 SSE updates stream
function listenToScraperProgress(onProgressCallback) {
    const eventSource = new EventSource(`${API_BASE_URL}/api/events`);

    eventSource.onmessage = (event) => {
        const payload = JSON.parse(event.data);
        console.log("Persona V3 Live Update:", payload);
        onProgressCallback(payload);
    };

    eventSource.onerror = (err) => {
        console.error("SSE connection error:", err);
        eventSource.close();
    };

    return eventSource;
}

```

| Critical Persona V3 VPS Operational Rules



1. Mandatory Headless Mode

A Linux VPS operates without an X11 graphical display window. `headless=True` must always be set when calling `LinkedInScraper` on VPS. If set to `False`, Playwright will fail to launch Chromium and throw display errors.



2. Linux Process Cleanup Fix

In `app.py` line 126, `_kill_playwright_chromium()` executes Windows `wmic` & `taskkill` commands. On a Linux VPS, update process cleanup to use cross-platform `psutil` or Linux `pkill` to avoid execution exceptions.

Cross-Platform Process Cleanup Code

`app.py` (Cross-platform Process Cleaner for Persona V3)

```
def _kill_playwright_chromium():
    """Cross-platform Chromium process cleaner for Persona V3 (Windows & Linux)
    import os, sys, subprocess
    try:
        if sys.platform.startswith('win'):
            # Windows taskkill logic
            subprocess.run(['wmic', 'process', 'where',
                           "name='chrome.exe' and CommandLine like '%browser_data%",
                           'get', 'ProcessId'], capture_output=True, timeout=5)
        else:
            # Linux / VPS pkill logic
            subprocess.run(['pkill', '-f', 'chromium.*browser_data'], capture_o
    except Exception as ex:
        print(f"Non-fatal process cleanup warning: {ex}")
```